7.1 多提供者配置与 API 密钥

> **生成模型**：GPT-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~22,000 tokens，输出 ~4,600 tokens（本节）

很多人第一次配 OpenClaw，会先卡在一个很现实的问题上：模型到底该怎么接？是只配 Claude，还是 Claude + GPT 一起上？Gemini 要不要留一个？本地模型有没有必要？如果你人在国内，API Key 放哪里最稳，OAuth 又是什么，为什么有的人不用 Key 也能跑 Claude？这一节就把这些事一次讲清楚。

先说结论：OpenClaw 的模型配置思路其实不复杂。它不是逼你押宝某一家，而是把模型层当成一个可以替换、可以并联、也可以回退的资源池。你可以只有一个默认模型；也可以同时配上 Anthropic、OpenAI、Google、国产平台和本地模型，然后按任务、价格和网络情况来切。

7.1.1 为什么建议一开始就配多个提供者

很多新手喜欢只配一个模型，理由也很直接：省事。问题是，一旦这个模型碰到限流、账单异常、地区网络问题、上下文限制，整个 OpenClaw 就像被拔了电源。

多提供者配置的价值，主要有四个：

第一，稳定。Claude 很强，但不代表你永远能顺畅打通 Anthropic API；OpenAI 好用，但某些地区延迟波动会很明显；Gemini 在长上下文和多模态上有优势，但也不适合所有任务。把几家一起配上，等于给自己留后路。

第二，分工。不是所有任务都值得上最贵模型。比如写复杂代码、做架构分析，可以让 Claude 或 GPT-5 这类高质量模型扛主力；日常摘要、改写、信息抽取，完全可以交给便宜很多的模型。

第三，网络适配。国际模型和国产模型的网络环境差异很大。你在大陆机器上部署 OpenClaw，常见做法不是“只用一个最强模型”，而是“国内直连模型做主力，国际模型做保底”。

第四，未来可迁移。今天你喜欢某个模型，不代表三个月后还会继续喜欢。把配置抽象成 provider + model 之后，后面切换成本就很低。

7.1.2 OpenClaw 里和模型有关的几个核心概念

在继续之前，先把几个词说明白，不然后面很容易看晕。

提供者（Provider），就是 Anthropic、OpenAI、Google 这种模型服务方，也可以是你自己搭的兼容接口，比如本地 Ollama、vLLM、LM Studio。

模型 ID，就是具体调用哪个模型，例如 `anthropic/claude-sonnet-4-6` 、 `openai/gpt-4o` 、 `google/gemini-2.5-pro` 。前面是提供者，后面是模型名。

默认模型（primary），就是平时优先用谁。

回退模型（fallbacks），是主模型失败后按顺序补位的列表。这个概念我们下一节会重点展开。

Auth Profile，可以理解成“同一提供者下的一份认证档案”。比如你可能有两个 Anthropic API Key，或者一个 API Key 加一个 OAuth 登录账号，这些都可以被当成不同的认证入口。

OAuth，是一种授权流程。它和“把 API Key 直接贴进配置文件”不是一回事。API Key 更像你自己发的门禁卡；OAuth 更像你登录官方账号，然后授予 OpenClaw 代表你调用某些能力。

7.1.3 API Key 放哪儿最合适

最常见的做法有两种：

一是放环境变量。
二是在 `openclaw.json` 里引用环境变量。

这两步最好一起用。不要把真实 Key 硬编码到配置里，尤其别把它写进 Git 仓库，不然你迟早会被自己坑一次。

先看环境变量的写法：

```
export ANTHROPIC_API_KEY="sk-ant-..."
export OPENAI_API_KEY="sk-..."
export GEMINI_API_KEY="AIza..."
export OPENROUTER_API_KEY="sk-or-..."
export LOCAL_LLM_API_KEY="dummy"
```

如果你用的是 macOS 或 Linux，这些变量通常可以放进 `~/.zshrc` 、 `~/.bashrc` 或 systemd/launchd 的服务环境里。注意一点：如果 OpenClaw 是后台服务启动的，别只在当前终端里 `export` ，否则你以为配好了，重启服务后又失效。

然后在 `openclaw.json` 里引用：

```
{
  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-sonnet-4-6",
        fallbacks: ["openai/gpt-4o", "google/gemini-2.5-pro"]
      }
    }
  },
  models: {
    providers: {
      anthropic: {
        apiKey: "$ANTHROPIC_API_KEY"
      },
      openai: {
        apiKey: "$OPENAI_API_KEY"
      },
      google: {
        apiKey: "$GEMINI_API_KEY"
      }
    }
  }
}
```

这个写法的好处很实际：配置文件能进版本管理，密钥本身不进；迁移机器时，只要重新注入环境变量就行。

7.1.4 一个够用的多提供者基础配置

如果你现在还没概念，可以先抄一个比较通用的版本：Claude 做主模型，GPT 做第一回退，Gemini 做第二回退，本地模型留给低成本实验。

```
{
  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-sonnet-4-6",
        fallbacks: [
          "openai/gpt-4o",
          "google/gemini-2.5-pro",
          "local/qwen2.5-coder-14b"
        ]
      }
    }
  },
  models: {
    mode: "merge",
    providers: {
      anthropic: {
        apiKey: "$ANTHROPIC_API_KEY"
      },
      openai: {
        apiKey: "$OPENAI_API_KEY"
      },
      google: {
        apiKey: "$GEMINI_API_KEY"
      },
      local: {
        baseUrl: "http://127.0.0.1:11434/v1",
        apiKey: "$LOCAL_LLM_API_KEY",
        api: "openai-chat",
        models: [
          {
            id: "qwen2.5-coder-14b",
            name: "Qwen2.5 Coder 14B Local",
            input: ["text"],
            contextWindow: 32768,
            maxTokens: 8192,
            reasoning: false,
            cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 }
          }
        ]
      }
    }
```

```
    }
  }
```

这里的 `local` 本质上就是一个自定义提供者，走 OpenAI 兼容接口。只要你的本地服务能说这种协议，OpenClaw 不太在意它底层到底是 Ollama、vLLM 还是别的东西。

7.1.5 Anthropic Claude 怎么配

Claude 在 OpenClaw 里通常是高质量主力位，尤其适合长文本、复杂代码和工具调用比较密集的任务。

最简单的方式就是 API Key：

```
{
  models: {
    providers: {
      anthropic: {
        apiKey: "$ANTHROPIC_API_KEY"
      }
    }
  },
  agents: {
    defaults: {
      model: { primary: "anthropic/claude-sonnet-4-6" }
    }
  }
}
```

但对不少个人用户来说，更关心的是另一条路：我已经是 Claude Pro 或 Max 订阅用户了，能不能不用开发者 API Key，直接复用账号授权？答案是可以，OpenClaw 支持走 OAuth 或对应登录流程。

你可以把它理解成：OpenClaw 不再拿一串静态 Key 去调用模型，而是通过你已经购买的账号体系，获得一个授权会话。这样做的好处是，对已有订阅用户更顺手；坏处是，它不像 API Key 那样“拿来就能塞进任何地方”，认证状态可能过期，也更依赖交互式登录。

常见命令大致是这样：

```
openclaw models auth add
openclaw models auth login --provider anthropic --set-default
openclaw models status --probe
```

如果你是长期运行的服务器用户，我更建议 API Key；如果你是个人桌面环境、已经买了 Claude Pro/Max，而且主要在自己机器上用，OAuth 会更贴近真实使用习惯。

7.1.6 OpenAI GPT 怎么配

OpenAI 的配置最直接，通常也是很多人第二个会接的提供者。原因很简单：生态成熟，模型线完整，兼容接口多，很多第三方也在模仿它的协议。

基础配置如下：

```
{
  models: {
    providers: {
      openai: {
        apiKey: "$OPENAI_API_KEY"
      }
    }
  },
  agents: {
    defaults: {
      model: { primary: "openai/gpt-4o" }
    }
  }
}
```

如果你把 OpenAI 放在回退位，也很常见：

```
{
  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-sonnet-4-6",
        fallbacks: ["openai/gpt-4o", "openai/gpt-4.1-mini"]
      }
    }
  },
  models: {
    providers: {
      openai: {
        apiKey: "$OPENAI_API_KEY"
      }
    }
  }
}
```

有些用户已经订了 ChatGPT 或 Codex 类产品，也会碰到和 Claude 类似的问题：能不能用账号授权而不是开发者 API。OpenClaw 的认证体系里也考虑了这类场景，不过从稳定性和可控性来说，生产环境还是开发者 API Key 更干脆。

7.1.7 Google Gemini 怎么配

Gemini 很适合做一个“第三顺位但很有价值”的提供者。原因是它在长上下文、多模态和某些成本档位上挺有竞争力，而且和 Claude、GPT 的风格差异也比较明显。你把它放进回退链，经常能补到另两家不太舒服的场景。

```
{
  models: {
    providers: {
      google: {
        apiKey: "$GEMINI_API_KEY"
      }
    }
  },
  agents: {
    defaults: {
      model: { primary: "google/gemini-2.5-pro" }
    }
  }
}
```

如果你还想给图像或轻量任务单独指定型号，也可以后续再做更细的模型策略。当前阶段先记住一句话：Gemini 很适合当“覆盖面广、成本相对平衡”的备份位。

7.1.8 本地模型怎么接入

很多人看到“本地模型”这四个字，会误以为 OpenClaw 内置了一整套本地推理框架。其实不是。OpenClaw 更像一个总调度器，它默认希望你把本地模型暴露成一个兼容 API，然后它来统一调用。

最常见的本地接法有三类：

一类是 Ollama；
一类是 LM Studio；
一类是 vLLM 这类更偏服务端的推理网关。

只要能暴露 OpenAI 兼容端点，配置思路就差不多。

比如接本地 Ollama：

```
{
  models: {
    providers: {
      local: {
        baseUrl: "http://127.0.0.1:11434/v1",
        apiKey: "$LOCAL_LLM_API_KEY",
        api: "openai-chat",
        models: [
          {
            id: "llama3.1:8b",
            name: "Llama 3.1 8B (Ollama)",
            input: ["text"],
            contextWindow: 8192,
            maxTokens: 4096,
            reasoning: false,
            cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 }
          }
        ]
      }
    }
  },
  agents: {
    defaults: {
      model: { primary: "local/llama3.1:8b" }
    }
  }
}
```

这里的 `apiKey` 往往只是占位符。很多本地服务根本不校验它，但 OpenClaw 的配置结构通常还是保留这个字段，写个 `dummy` 也没问题。

本地模型最适合三种事：隐私要求高、频繁低成本调用、离线或弱网环境。它不一定是你最强的模型，但很可能是你最省钱、最自由的那张底牌。

7.1.9 Auth Profile 到底解决什么问题

这一块很容易被忽略，但在真正跑起来以后，你会发现它特别有用。

假设你只有一个 OpenAI API Key。那很简单，能用就用，挂了就报错。可一旦你有多个凭据，比如：

- 一个公司报销的 API Key

- 一个你自己的备用 Key
- 一个 OAuth 登录档案

这时候 OpenClaw 就不需要把所有请求都死死绑在单一凭据上。它可以维护多个 Auth Profile，优先用默认的，失败了再轮转。

这和“回退到另一个模型”是两码事。Auth Profile 轮换，还是同一个模型；只是换凭据。比如 `anthropic/claude-sonnet-4-6` 不变，但认证入口从 profile A 切到 profile B。

你可以把它理解成三级兜底里的内层保险：

1. 先试主模型的默认凭据
2. 再试主模型的其他凭据
3. 实在不行，再退到别的模型

这个设计特别适合团队共用部署、多个账号混合、或者个人用户有订阅账号又有 API Key 的场景。

7.1.10 一个更像生产环境的配置模板

如果你希望一开始就配得像样一点，可以参考下面这个版本：

```
{
  agents: {
    defaults: {
      model: {
        primary: "anthropic/claude-sonnet-4-6",
        fallbacks: [
          "openai/gpt-4o",
          "google/gemini-2.5-pro",
          "local/qwen2.5-coder-14b"
        ]
      }
    }
  },
  models: {
    mode: "merge",
    providers: {
      anthropic: {
        apiKey: "$ANTHROPIC_API_KEY"
      },
      openai: {
        apiKey: "$OPENAI_API_KEY"
      },
      google: {
        apiKey: "$GEMINI_API_KEY"
      },
      local: {
        baseUrl: "http://127.0.0.1:11434/v1",
        apiKey: "$LOCAL_LLM_API_KEY",
        api: "openai-chat",
        models: [
          {
            id: "qwen2.5-coder-14b",
            name: "Qwen2.5 Coder 14B Local",
            input: ["text"],
            contextWindow: 32768,
            maxTokens: 8192,
            reasoning: false,
            cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 }
          }
        ]
      }
    }
```

```
    }
  }
```

这份配置背后的思路很朴素：

- 日常高质量任务优先 Claude
- Claude 异常时交给 GPT
- 再不行交给 Gemini
- 最后至少还有本地模型兜底

你不一定非得按这个顺序来，但这个结构本身很值得参考。

7.1.11 配置时最容易踩的坑

最后说几个很常见的坑。

第一，把真实 Key 直接写进 `openclaw.json` 。短期看方便，长期看就是等着泄漏。

第二，只在当前 shell 里导出变量，忘了后台服务的运行环境。命令行能跑，不代表守护进程也能跑。

第三，把 OAuth 当成永久凭据。它更像登录状态，不是万年不变的静态密钥。

第四，本地模型接口不兼容却硬套 OpenAI 配置。表面是“连接成功”，实际一调用就炸。遇到这种情况，先确认端点路径、协议模式和模型名是不是对得上。

第五，所有模型都配了，但没有明确主模型和回退顺序。结果看上去很多，真出问题时却不知道到底会落到谁身上。

本节小结

这节的重点不是“把所有模型都接上”，而是建立一个能长期使用的配置思路：密钥放环境变量， `openclaw.json` 负责引用；主模型和回退模型分清楚；API Key 和 OAuth 分场景使用；国际模型、本地模型、甚至后面要讲的国产模型，都放进同一套 provider 配置框架里。这样你后面再谈成本、回退链和国内接入，才不会越配越乱。